capital

Custom Bridge Processor Development

Software Version 2015.1

February 2016

© 2016 Mentor Graphics Corporation
All rights reserved.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

RESTRICTED RIGHTS LEGEND 03/97

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202-3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

Contractor/manufacturer is:
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: www.mentor.com/supportnet
Contact Your Technical Writer: www.mentor.com/supportnet/documentation/reply_form.cfm

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other third parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the respective third-party owner. The use herein of a third-party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: www.mentor.com/terms_conditions/trademarks.cfm.

End-User License Agreement: You can print a copy of the End-User License Agreement from: www.mentor.com/terms_conditions/enduser.cfm.

Table Of Contents

Introduction **2**

Interfaces for Bridge Processing **4**

IXBridgeProcessor 4
IXHarnessBridgeProcessor 5
IXIntegratorBridgeProcessor 5
IXTopologyBridgeProcessor 5
IXLogicBridgeProcessor 5

Example for Bridge Processing **6**

BridgeProcessor 6

Interfaces for Bridge Import Pre-Processing **11**

IXBridgePrePostProcessor 11
IXBridgeImportPreProcessor 12

Example for Bridge Import Pre-Processing **14**

KBLApprovalElementImportProcessor 14

Interfaces for Bridge Export Post-Processing **20**

IXBridgePrePostProcessor 20
IXBridgeExportPostProcessor 20

Example for Bridge Export Post-Processing **22**

DSIConnectorUserPropertyExportProcessor 22

Introduction

Bridges supports exchange of data between Capital and a wide variety of systems through its adaptor framework. One may write a new adaptor to suit one's requirement and plug it into the Capital environment for use. Also, Capital provides some in-built adaptors for users to work with existing standard data formats like DSI, KBL, PLMXML, etc.

Bridge Out has two stages.

1. Convert Capital data to HX2ML data
2. Convert HX2ML data to Adaptor data

Similarly, Bridge In has two stages.

1. Convert Adaptor data to HX2ML data
2. Convert HX2ML data to Capital data

Because of a variety of requirements to process the data differently, we have IXBridgeProcessor, which acts on the X2ML or HX2ML data and modifies it as per the user requirement. This may be used either during Bridge Out or during Bridge In or both. Please note that the IXBridgeProcessor plugin changes are applied for any data format because every data format is converted into X2ML or HX2ML during Bridge In/Out. So, if the requirement is to process the data irrespective of the data format being imported/exported, then IXBridgeProcessor is the plugin to be used.

IXBridgePrePostProcessor enables the user to act on the adaptor data directly. This may be required when

- Not all data in the adaptor file is supported by the existing adaptor
- IXBridgeProcessor may not be useful because the unsupported data is not available in the HX2ML format during Bridge out / Bridge In

IXBridgePrePostProcessor will act directly on the adaptor data and modify it in a way such that it is bridged into/out of Capital in a desired way.

IXBridgeImportPreProcessor enables the user to modify the adaptor data during Bridge In.

IXBridgeExportPostProcessor enables the user to modify the adaptor data during Bridge Out.

Figure 1 – The context of various bridge processors

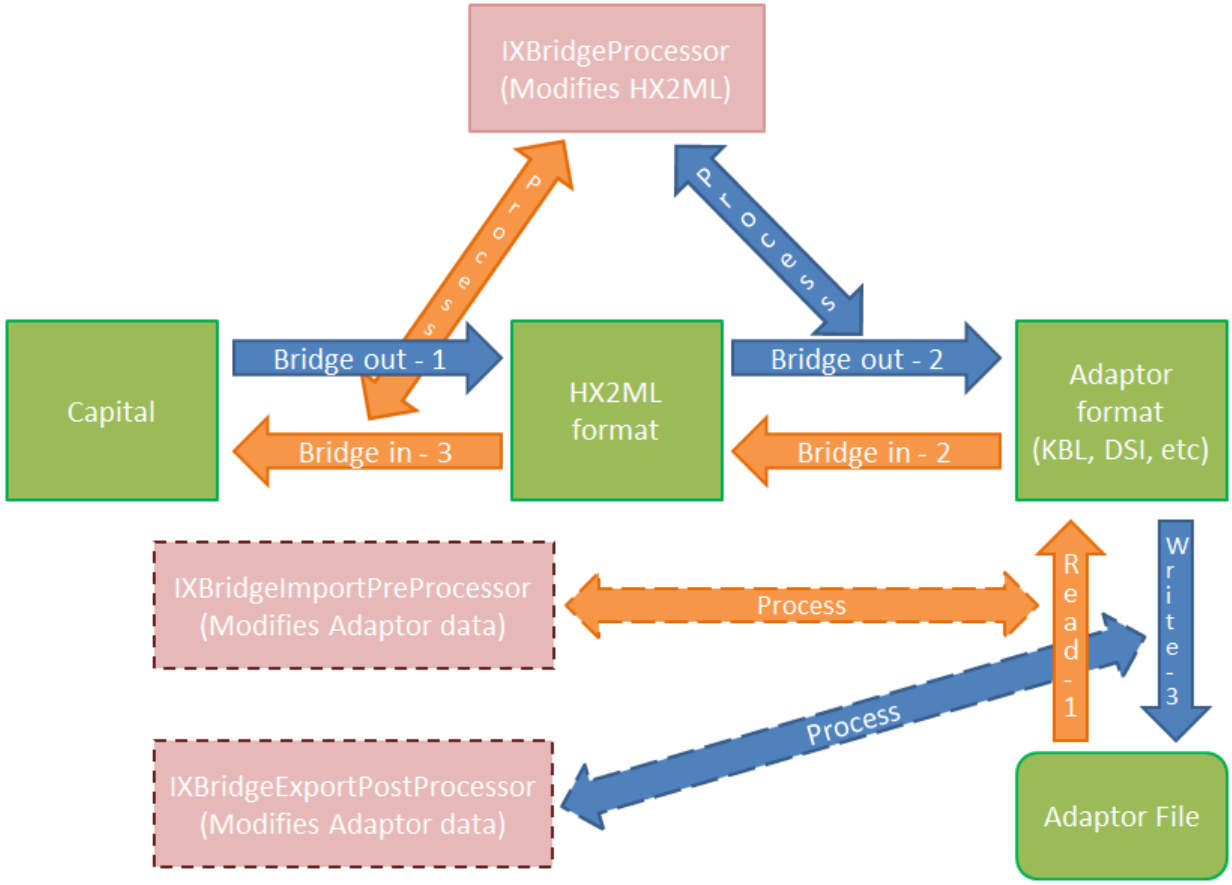
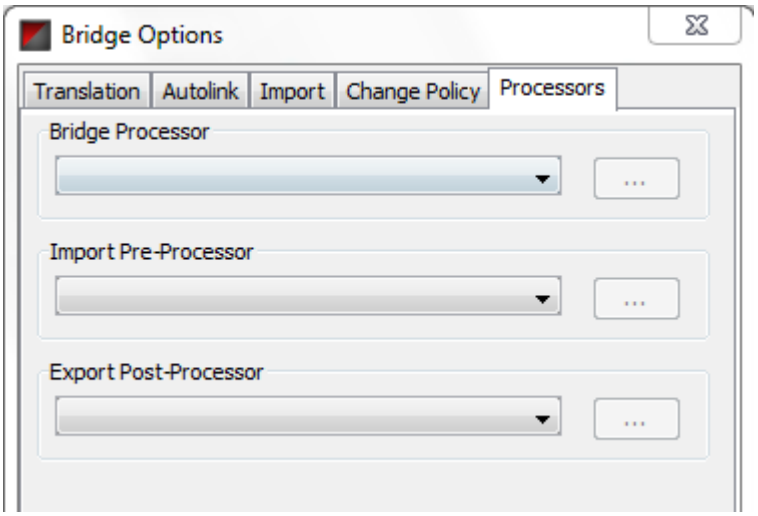


Figure 2 – Bride Options UI to set and use the various Bridge Processors



Interfaces for Bridge Processing

IXBridgeProcessor

This is the main interface for Bridge Processing. But, this is not to be implemented directly. Appropriate sub interfaces need to be implemented.

1. Implement IXHarnessBridgeProcessor for processing during Bridge In/Out into/from Capital Harness XC or Capital Modular XC
2. Implement IXIntegratorBridgeProcessor for processing during Bridge In/Out into/from Capital Integrator
3. Implement IXTopologyBridgeProcessor for processing during Bridge In/Out into/from Capital Topology
4. Implement IXLogicBridgeProcessor for processing during Bridge In/Out into/from Capital Logic

Methods in this interface are

1. void processBridgeIn(Object x2mlContainer, IXApplicationContext applicationContext)
    a. Given an x2ml container instance, this method is called to allow any type of modification of this container so that these changes are considered during Bridge In
    b. “x2mlContainer” is the X2ML container created during import to application. It can be either chs.bridges.x2ml.WiringDesign if it is a wiring bridge or chs.bridges.x2ml.harness.Harness if it is a harness bridge
    c. “applicationContext” is the application context. From this object, the current "bridge context" IXBridgeContext may be obtained. Also, the current design, project and currently selected objects may be obtained. If the Bridge Processor is invoked from a non-design application like Capital Project, then the current design, current diagram, current project etc. are not initialised and have a value of null
2. void processBridgeOut(Object x2mlContainer, IXApplicationContext applicationContext)
    a. Given an x2ml container instance, this method is called to allow any type of modification of this container so that these changes are considered during Bridge Out
    b. “x2mlContainer” is the X2ML container created during export from application. It can be either chs.bridges.x2ml.WiringDesign if it is a wiring bridge or chs.bridges.x2ml.harness.Harness if it is a harness bridge
    c. “applicationContext” is the application context. From this object, the current "bridge context" IXBridgeContext may be obtained. Also, the current design, project and currently selected objects may be obtained. If the Bridge Processor is invoked from a non-design application like Capital Project, then the current design, current diagram, current project etc. are not initialised and have a value of null

IXHarnessBridgeProcessor

This is the interface to be implemented for processing during Bridge In/Out into/from Capital Harness XC or Capital Modular XC. This does not contain any methods.

IXIntegratorBridgeProcessor

This is the interface to be implemented for processing during Bridge In/Out into/from Capital Integrator. This does not contain any methods.

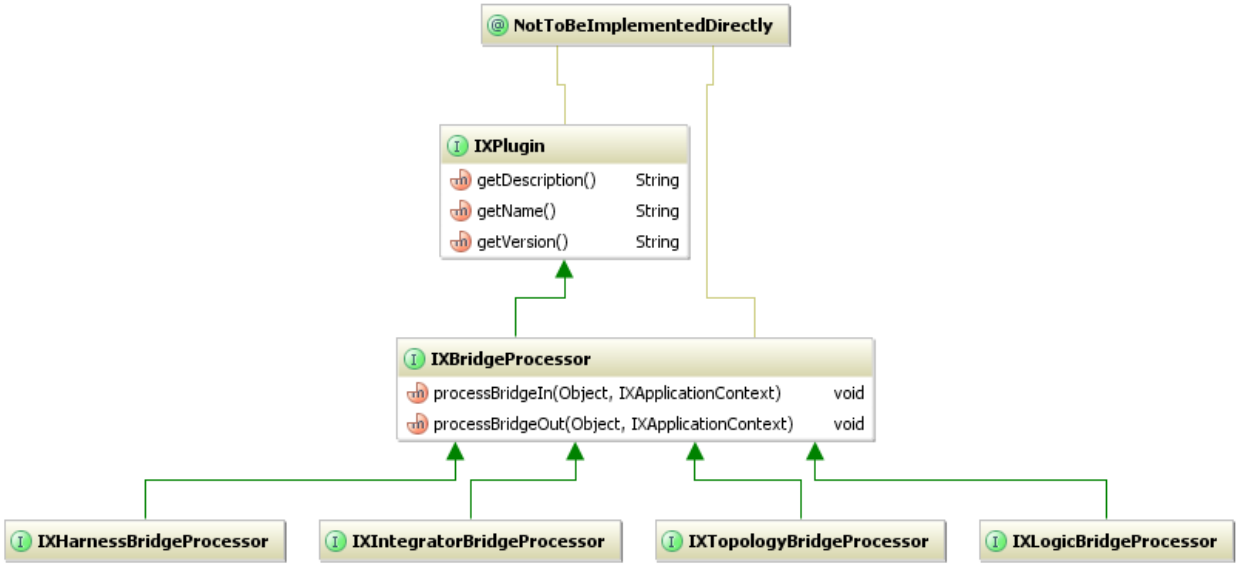
IXTopologyBridgeProcessor

This is the interface to be implemented for processing during Bridge In/Out into/from Capital Topology. This does not contain any methods.

IXLogicBridgeProcessor

This is the interface to be implemented for processing during Bridge In/Out into/from Capital Logic. This does not contain any methods.

Figure 3 – Class hierarchy of Bridge Processors



Example for Bridge Processing

We shall consider an example where

1. During Bridge In
    a. If the “wirecolor” attribute value on each “wire” element matches with the description of a “color code” in the Capital Library, then the “color code” value is set as the “wirecolor” value. The existing “wirecolor” value is added as a user property on the wire, with name “ColorBeforeBridgeInTranslation”
2. During Bridge Out
    a. The description of the “color code” in Capital Library is set as the “wirecolor” attribute on each wire. The actual “color code” value is added as a user property on the wire, with name “ColorBeforeBridgeOutTranslation”

The java file for this example is available at
<capital_home>\doc\plugin\examples\Java\src\com\example\plugin\bridge\BridgeProcessor.java

BridgeProcessor

```
/**
 * Copyright 2009 Mentor Graphics Corporation. All Rights Reserved.
 * <p>
 * Recipients who obtain this code directly from Mentor Graphics use it solely
 * for internal purposes to serve as example plugin.
 * This code may not be used in a commercial distribution. Recipients may
 * duplicate the code provided that all notices are fully reproduced with
 * and remain in the code. No part of this code may be modified, reproduced,
 * translated, used, distributed, disclosed or provided to third parties
 * without the prior written consent of Mentor Graphics, except as expressly
 * authorized above.
 * <p>
 * THE CODE IS MADE AVAILABLE "AS IS" WITHOUT WARRANTY OR SUPPORT OF ANY KIND.
 * MENTOR GRAPHICS OFFERS NO EXPRESS OR IMPLIED WARRANTIES AND SPECIFICALLY
 * DISCLAIMS ANY WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR
PURPOSE,
 * OR WARRANTY OF NON-INFRINGEMENT. IN NO EVENT SHALL MENTOR GRAPHICS OR ITS
 * LICENSORS BE LIABLE FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR
CONSEQUENTIAL
 * DAMAGES (INCLUDING LOST PROFITS OR SAVINGS) WHETHER BASED ON CONTRACT, TORT
 * OR ANY OTHER LEGAL THEORY, EVEN IF MENTOR GRAPHICS OR ITS LICENSORS HAVE
BEEN
```

```
 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.
 * <p>
 */
package com.example.plugin.bridge;

import com.mentor.chs.plugin.changemanager.IXHarnessBridgeProcessor;
import com.mentor.chs.plugin.changemanager.IXIntegratorBridgeProcessor;
import com.mentor.chs.plugin.changemanager.IXLogicBridgeProcessor;
import com.mentor.chs.plugin.IXApplicationContext;
import com.mentor.chs.plugin.IXApplicationContextListener;
import com.mentor.chs.api.IXLibraryColorCode;
import chs.bridges.x2ml.UserProperty;
import chs.bridges.x2ml.WiringDesign;
import chs.bridges.x2ml.Wire;
import chs.bridges.x2ml.harness.Harness;
import chs.bridges.adaptors.IAdaptorReporter;

import java.util.Collection;
import java.util.HashSet;

/**
 * Sample plugin for Bridge processing.
 */
public class BridgeProcessor implements IXHarnessBridgeProcessor,
IXIntegratorBridgeProcessor, IXLogicBridgeProcessor,
            IXApplicationContextListener
{

      /**
       * Application context
       */
      private IXApplicationContext m_appContext;

      /**
       * Return the description of the plugin
       * @return String type - description of the plugin
       */
      public String getDescription() {
            return "Demonstrate bridge processing. Color code translation
between code name-description & add properties";
      }

      /**
```

```
       * Return the name of the plugin
       * @return String type - name of the plugin
       */
      public String getName() {
            return "Bridge Processor Example";
      }

      /**
       * Return the version of the plugin
       * @return String type - version of the plugin
       */
      public String getVersion() {
            return "1.0";
      }

      public void processBridgeIn(Object x2mlContainer, IXApplicationContext
applicationContext)
      {
            Collection wireColl = null;
            if(x2mlContainer instanceof Harness){
                  Harness incHarn = (Harness)x2mlContainer;
                  wireColl = incHarn.getElementsByType(Wire.getElementName());
            }
            else if(x2mlContainer instanceof WiringDesign){
                  WiringDesign des = (WiringDesign)x2mlContainer;
                  wireColl = des.getElementsByType(Wire.getElementName());
            }
            for(Object elem : wireColl){
                  if(elem instanceof Wire){
                        Wire wire = (Wire) elem;
                        String color = wire.getColor();
                        if(color != null && !color.trim().isEmpty()){
                              boolean found = false;
                              for(IXLibraryColorCode libcolor :
applicationContext.getLibrary().getColorCodes()){

if(libcolor.getAttribute("Description").equalsIgnoreCase(color)){
                                          found = true;

wire.setColor(libcolor.getAttribute("ColorCode"));
                                          break;
                                    }
                              }
```

```
                              if (!found && m_appContext != null &&
m_appContext.getBridgeContext() != null) {
                                    IAdaptorReporter reporter =

      (IAdaptorReporter)m_appContext.getBridgeContext().getAdaptorReporter();
                                    if (reporter != null) {

      reporter.logMessage(IAdaptorReporter.WARNING,
                                                          new
StringBuilder().append("Color with description '").append(color)
                                                                        .append("'
is not present in the library").toString(),
                                                          "Add a valid color to
the wire (This is reported by the Bridge processor plugin)");
                                    }
                              }
                              wire.addUserProperty(new
UserProperty("ColorBeforeBridgeInTranslation",color));
                        }
                  }
            }
      }

      public void processBridgeOut(Object x2mlContainer, IXApplicationContext
applicationContext)
      {
            Collection wireColl = null;
            if(x2mlContainer instanceof Harness){
                  Harness incHarn = (Harness)x2mlContainer;
                  wireColl = incHarn.getElementsByType(Wire.getElementName());
            }
            else if(x2mlContainer instanceof WiringDesign){
                  WiringDesign des = (WiringDesign)x2mlContainer;
                  wireColl = des.getElementsByType(Wire.getElementName());
            }
            Collection<String> reportedColors = new HashSet<String>();
            for(Object elem : wireColl){
                  if(elem instanceof Wire){
                        Wire wire = (Wire) elem;
                        String color = wire.getColor();
                        if(color != null && !color.trim().isEmpty()){
                              for(IXLibraryColorCode libcolor :
applicationContext.getLibrary().getColorCodes()){
```

```
if(libcolor.getAttribute("ColorCode").equalsIgnoreCase(color)){

wire.setColor(libcolor.getAttribute("Description"));
                                        break;
                                    }
                                }
                                if (m_appContext != null &&
m_appContext.getBridgeContext() != null &&
                                            !reportedColors.contains(color)) {
                                        IAdaptorReporter reporter =

      (IAdaptorReporter)m_appContext.getBridgeContext().getAdaptorReporter();
                                        if (reporter != null) {

      reporter.logMessage(IAdaptorReporter.WARNING,
                                                                  new
StringBuilder().append("Color code '").append(color)
                                                                                .append("'
is translated to '").append(wire.getColor())
                                                                                .append("\'
by the bridge processor").toString(), null);
                                            reportedColors.add(color);
                                        }
                                }
                                wire.addUserProperty(new
UserProperty("ColorBeforeBridgeOutTranslation",color));
                            }
                        }
            }
      }

      public void setApplicationContext(IXApplicationContext
applicationContext)
      {
            m_appContext = applicationContext;
      }
}
```

Interfaces for Bridge Import Pre-Processing

IXBridgePrePostProcessor

This is the main interface for Bridge In pre-processing and Bridge Out post-processing. But, this is not to be implemented directly. The plugins should implement IXBridgeImportPreProcessor for Bridge In pre-processing. Along with this, depending upon in which applications the import processor should be available, the plugin needs to implement one or more of the below interfaces.

1. Implement IXHarnessPlugin for processing during Bridge In into Capital Harness XC or Capital Modular XC
2. Implement IXIntegratorPlugin for processing during Bridge In into Capital Integrator
3. Implement IXTopologyPlugin for processing during Bridge In into Capital Topology
4. Implement IXLogicPlugin for processing during Bridge In into Capital Logic

Methods in this interface are

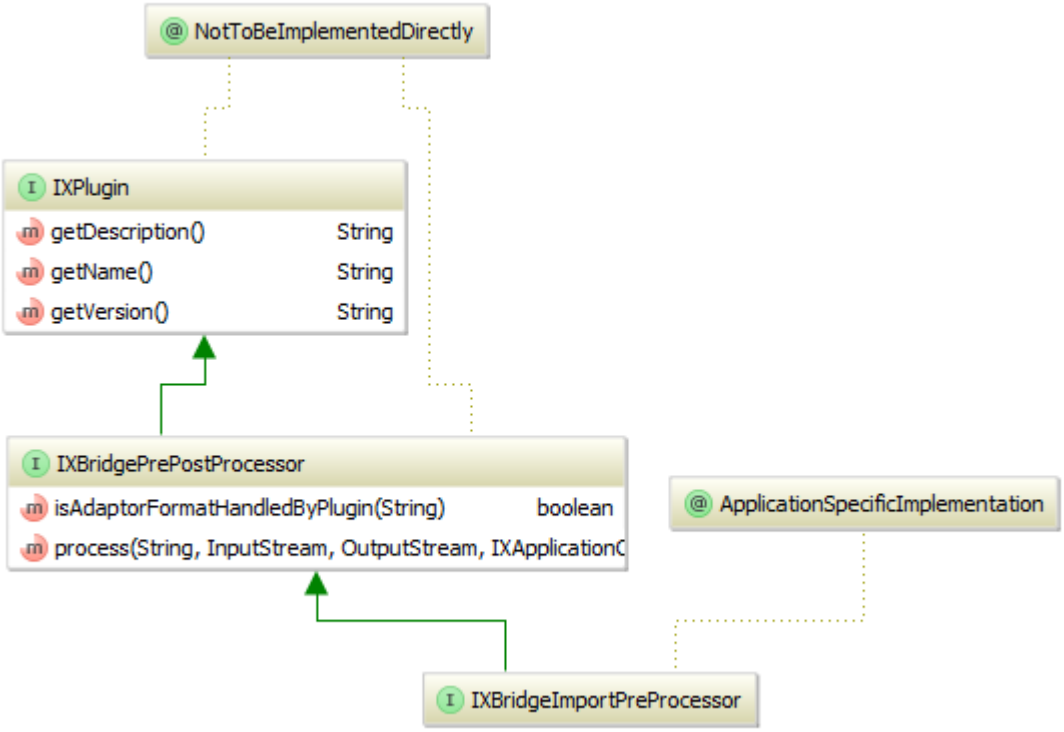
1. boolean isAdaptorFormatHandledByPlugin(String adaptorFormatDescription)
    a. Indicates whether this processor handles the given adaptor format
    b. Return ‘true’ if the plugin can handle the given adaptor format; false otherwise
    c. “adaptorFormatDescription” is the format description of the adaptor data to be processed. Refer to "<capital_home>/adaptors/adaptors.xml" file section "1-Adaptors declarations" for the description string of each format. For example, "DSI Harness", "KBL Adaptor", etc. This processor is not recommended for 'X2ML' and 'HX2ML' formats. If processing is required for 'X2ML' or 'HX2ML' format data, then use appropriate sub-class of IXBridgeProcessor
2. boolean process(String adaptorFormatDescription, InputStream adaptorData, OutputStream processedAdaptorData, IXApplicationContext applicationContext) throws Exception
    a. Processes or modifies the adaptor data that is bridged in/out. This is invoked only if “isAdaptorFormatHandledByPlugin(String)” returns true for the given adaptor format
    b. Return true if 'adaptorData' is processed and the processed data is written into 'processedAdaptorData'; false otherwise. If 'true' is returned, then the content of 'processedAdaptorData' is used for Bridge In or Bridge Out. If 'false' is returned, then the original content of 'adaptorData' is used for Bridge In or Bridge Out
    c. “adaptorFormatDescription” is the format description of the adaptor data to be processed. Refer to "<capital_home>/adaptors/adaptors.xml" file section "1-Adaptors declarations" for the description string of each format. For example, "DSI Harness", "KBL Adaptor", etc. This processor is not recommended for 'X2ML' and 'HX2ML' formats. If processing is required for 'X2ML' or 'HX2ML' format data, then use appropriate sub-class of IXBridgeProcessor
    d. “adaptorData” the original adaptor data stream to be bridged in/out without any modification

e. “processedAdaptorData” the adaptor data stream which is to contain the processed data by this processor
f. “applicationContext” is the application context. From this object, the current "bridge context" IXBridgeContext may be obtained. Also, the current design, project and currently selected objects may be obtained. If the Import Pre-Processor or the Export Post-Processor is invoked from a non-design application like Capital Project, then the current design, current diagram, current project etc. are not initialised and have a value of null
g. Throws any exception that occurs during the processing of adaptor data

IXBridgeImportPreProcessor

This is the main interface for Bridge In pre-processing and does not contain any methods. But, this interface should be implemented along with the application specific interfaces. Refer to IXBridgePrePostProcessor

Figure 3 – Class hierarchy of Bridge Import Pre-Processors



Example for Bridge Import Pre-Processing

We shall consider an example where the user is interested in the “Approval” elements in a KBL file. The KBL adaptor ignores these elements. So, there is no provision for the user to access them in Capital. This issue may be addressed by writing an appropriate Bridge Import Pre-Processor plugin.

The following sample converts all “Approval” elements in a KBL file into “Processing_information” elements under the “Harness” element of the KBL. Since all the “Processing_information” elements of “Harness” element are converted to user properties of the harness in Capital, user has an access to the information in “Approval” elements in Capital, via these user properties on the harness.

The java file for this example is available at
<capital_home>\doc\plugin\examples\Java\src\com\example\plugin\bridge\
KBLApprovalElementImportProcessor.java

KBLApprovalElementImportProcessor

```
/**
 * Copyright 2014 Mentor Graphics Corporation. All Rights Reserved.
 * <p>
 * Recipients who obtain this code directly from Mentor Graphics use it solely
 * for internal purposes to serve as example plugin.
 * This code may not be used in a commercial distribution. Recipients may
 * duplicate the code provided that all notices are fully reproduced with
 * and remain in the code. No part of this code may be modified, reproduced,
 * translated, used, distributed, disclosed or provided to third parties
 * without the prior written consent of Mentor Graphics, except as expressly
 * authorized above.
 * <p>
 * THE CODE IS MADE AVAILABLE "AS IS" WITHOUT WARRANTY OR SUPPORT OF ANY KIND.
 * MENTOR GRAPHICS OFFERS NO EXPRESS OR IMPLIED WARRANTIES AND SPECIFICALLY
 * DISCLAIMS ANY WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR
PURPOSE,
 * OR WARRANTY OF NON-INFRINGEMENT. IN NO EVENT SHALL MENTOR GRAPHICS OR ITS
 * LICENSORS BE LIABLE FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR
CONSEQUENTIAL
 * DAMAGES (INCLUDING LOST PROFITS OR SAVINGS) WHETHER BASED ON CONTRACT, TORT
 * OR ANY OTHER LEGAL THEORY, EVEN IF MENTOR GRAPHICS OR ITS LICENSORS HAVE
BEEN
```

```
 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.
 * <p>
 */
package com.example.plugin.bridge;

import com.mentor.chs.plugin.IXApplicationContext;
import com.mentor.chs.plugin.IXHarnessPlugin;
import com.mentor.chs.plugin.IXIntegratorPlugin;
import com.mentor.chs.plugin.IXLogicPlugin;
import com.mentor.chs.plugin.IXTopologyPlugin;
import com.mentor.chs.plugin.changemanager.IXBridgeImportPreProcessor;
import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.Node;
import org.w3c.dom.NodeList;
import org.xml.sax.SAXException;

import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException;
import javax.xml.transform.Result;
import javax.xml.transform.Source;
import javax.xml.transform.TransformerException;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.util.HashMap;
import java.util.Map;

/**
 * Sample plugin to demonstrate bridge import pre processing.
 * <p/>
 * For each Approval element in a KBL file, a user property is created on the
Harness with the Approval name as the user
 * property name and Approval type as the user property value
 */
public class KBLApprovalElementImportProcessor
            implements IXBridgeImportPreProcessor, IXHarnessPlugin,
IXIntegratorPlugin, IXTopologyPlugin, IXLogicPlugin
{
```

```
	private static final String KBL_ROOT = "kbl:KBL_container";
	private static final String APPROVAL = "Approval";
	private static final String NAME = "Name";
	private static final String TYPE_OF_APPROVAL = "Type_of_approval";
	private static final String HARNESS = "Harness";
	private static final String PROCESSING_INFORMATION =
"Processing_information";
	private static final String ID = "id";
	private static final String INSTRUCTION_TYPE = "Instruction_type";
	private static final String INSTRUCTION_VALUE = "Instruction_value";
	private static final String KBL = "KBL";

	/**
	 * Return the description of the plugin
	 *
	 * @return String type - description of the plugin
	 */
	public String getDescription()
	{
		return "For each Approval element in KBL, a user property is
created on the Harness for reference in Capital";
	}

	/**
	 * Return the name of the plugin
	 *
	 * @return String type - name of the plugin
	 */
	public String getName()
	{
		return "KBL Approval Element Import Processor";
	}

	/**
	 * Return the version of the plugin
	 *
	 * @return String type - version of the plugin
	 */
	public String getVersion()
	{
		return "1.0";
	}
```

```
	@Override
	public boolean isAdaptorFormatHandledByPlugin(String
adaptorFormatDescription)
	{
		return adaptorFormatDescription.contains(KBL);
	}

	@Override
	public boolean process(String adaptorFormatDescription, InputStream
adaptorData,
			OutputStream processedAdaptorData, IXApplicationContext
applicationContext) throws Exception
	{
		// create document builder
		DocumentBuilder builder = createDocumentBuilder();
		// create the DOM from input stream
		Document domdoc = getDOMDocument(adaptorData, builder);
		// add user properties from Approval elements in KBL
		boolean dataModified =
addApprovalElementsAsUserProperties(domdoc);
		// modify the stream by adding user properties
		if (dataModified) {
			processOutputStream(domdoc, processedAdaptorData);
		}
		return dataModified;
	}

	private void processOutputStream(Document domdoc, OutputStream
processedAdaptorData) throws TransformerException
	{
		Source xmlSource = new DOMSource(domdoc);
		Result outputTarget = new StreamResult(processedAdaptorData);

	TransformerFactory.newInstance().newTransformer().transform(xmlSource,
outputTarget);
	}

	private boolean addApprovalElementsAsUserProperties(Document domdoc)
	{
		if (domdoc != null) {
			Element documentElement = domdoc.getDocumentElement();
```

```
                  if (documentElement != null &&
KBL_ROOT.equals(documentElement.getNodeName())) {
                        Map<String, String> approvals =
getApprovalElements(documentElement);
                        return addUserPropertiesToHarness(documentElement,
approvals);
                  }
            }
            return false;
      }

      private boolean addUserPropertiesToHarness(Element documentElement,
Map<String, String> approvals)
      {
            boolean modified = false;
            NodeList harnessList =
documentElement.getElementsByTagName(HARNESS);
            if (harnessList.getLength() > 0) {
                  Node harness = harnessList.item(0);
                  Document ownerDocument = harness.getOwnerDocument();

                  int i = 0;
                  for (String approvalName : approvals.keySet()) {
                        i++;
                        Element procIns =
ownerDocument.createElement(PROCESSING_INFORMATION);
                        procIns.setAttribute(ID, APPROVAL + "_" + i);
                        harness.appendChild(procIns);

                        Element insType =
ownerDocument.createElement(INSTRUCTION_TYPE);
                        insType.setTextContent(approvalName);
                        procIns.appendChild(insType);

                        Element insValue =
ownerDocument.createElement(INSTRUCTION_VALUE);
                        String approvalValue = approvals.get(approvalName);
                        insValue.setTextContent(approvalValue);
                        procIns.appendChild(insValue);

                        System.out.println("Log from " + getName() + " - Added
user property with name = \"" + approvalName +
```

```
                                    "\" and value = \"" + approvalValue +
"\"");
                }
                modified = i > 0;
            }
            return modified;
        }

        private Map<String, String> getApprovalElements(Element documentElement)
        {
            NodeList approvalList =
documentElement.getElementsByTagName(APPROVAL);
            Map<String, String> approvals = new HashMap<String, String>();
            for (int i = 0; i < approvalList.getLength(); i++) {
                Node approval = approvalList.item(i);
                NodeList childNodes = approval.getChildNodes();
                String approvalName = null;
                String approvalType = null;
                for (int j = 0; j < childNodes.getLength(); j++) {
                    Node child = childNodes.item(j);
                    if (child.getNodeName().equals(NAME)) {
                        approvalName = child.getTextContent();
                    }
                    else if (child.getNodeName().equals(TYPE_OF_APPROVAL))
{
                        approvalType = child.getTextContent();
                    }
                }
                if (approvalName != null && approvalType != null) {
                    approvals.put(approvalName, approvalType);
                }
            }
            return approvals;
        }

        private Document getDOMDocument(InputStream adaptorInput,
DocumentBuilder builder) throws IOException, SAXException
        {
            Document domdoc = null;
            if (builder != null) {
                domdoc = builder.parse(adaptorInput);
            }
            return domdoc;
```

```
	}

	private DocumentBuilder createDocumentBuilder() throws
ParserConfigurationException
	{
		DocumentBuilderFactory factory =
DocumentBuilderFactory.newInstance();
		DocumentBuilder builder = factory.newDocumentBuilder();
		return builder;
	}
}
```

Interfaces for Bridge Export Post-Processing

IXBridgePrePostProcessor

This is the main interface for Bridge In pre-processing and Bridge Out post-processing. But, this is not to be implemented directly. The plugins should implement IXBridgeExportPostProcessor for Bridge Out post-processing. Along with this, depending upon in which applications the export processor should be available, the plugin needs to implement one or more of the below interfaces.

1. Implement IXHarnessPlugin for processing during Bridge Out from Capital Harness XC or Capital Modular XC
2. Implement IXIntegratorPlugin for processing during Bridge Out from Capital Integrator
3. Implement IXTopologyPlugin for processing during Bridge Out from Capital Topology
4. Implement IXLogicPlugin for processing during Bridge Out from Capital Logic

See “Interfaces for Bridge Import Pre-Processing” section for a description of the methods in this interface.

IXBridgeExportPostProcessor

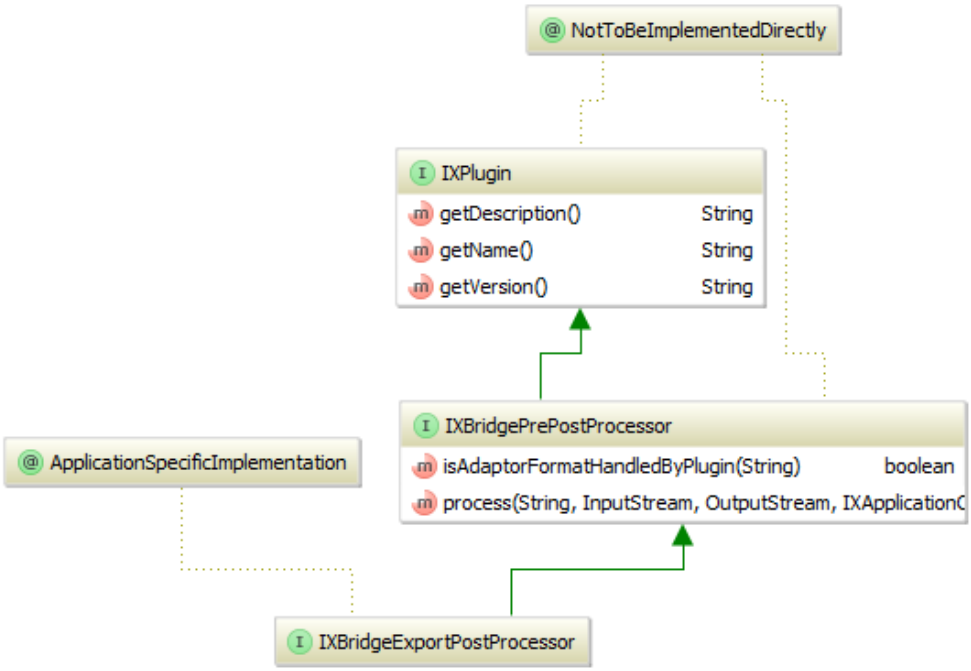
This is the main interface for Bridge Out post-processing and does not contain any methods.. But, this interface should be implemented along with the application specific interfaces. Refer to “IXBridgePrePostProcessor

Bridge out of some formats exports a zip file (like HCV, wiring DSI). For such cases, the post processor plugin needs to handle the input and output streams accordingly. The following are the steps:

1. Create a ZipInputStream using input stream argument and iterate through each zip entry as required. For example, in an HCV, there would be a KBL, an SVG and an index.xml; for a wiring DSI, each entry would be a DSI

2. Process the entries as required and add them to a ZipOutputStream which is created using the output stream argument passed to the plugin

Figure 3 – Class hierarchy of Bridge Export Post-Processors



Example for Bridge Export Post-Processing

We shall consider an example where the user is interested to know the user properties on a connector, in the exported DSI file. DSI format does not support exchange of user properties on an object. But using an appropriate Bridge export post-processor, user can achieve this by writing the user properties on a connector as “comment” records in the exported DSI file.

The following example writes the user properties on a design connector as “comment” records in the exported DSI file.

The java file for this example is available at
<capital_home>\doc\plugin\examples\Java\src\com\example\plugin\bridge\
DSIConnectorUserPropertyExportProcessor.java

DSIConnectorUserPropertyExportProcessor

```
/**
 * Copyright 2014 Mentor Graphics Corporation. All Rights Reserved.
 * <p>
 * Recipients who obtain this code directly from Mentor Graphics use it solely
 * for internal purposes to serve as example plugin.
 * This code may not be used in a commercial distribution. Recipients may
 * duplicate the code provided that all notices are fully reproduced with
 * and remain in the code. No part of this code may be modified, reproduced,
 * translated, used, distributed, disclosed or provided to third parties
 * without the prior written consent of Mentor Graphics, except as expressly
 * authorized above.
 * <p>
 * THE CODE IS MADE AVAILABLE "AS IS" WITHOUT WARRANTY OR SUPPORT OF ANY KIND.
 * MENTOR GRAPHICS OFFERS NO EXPRESS OR IMPLIED WARRANTIES AND SPECIFICALLY
 * DISCLAIMS ANY WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR
PURPOSE,
 * OR WARRANTY OF NON-INFRINGEMENT. IN NO EVENT SHALL MENTOR GRAPHICS OR ITS
 * LICENSORS BE LIABLE FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR
CONSEQUENTIAL
 * DAMAGES (INCLUDING LOST PROFITS OR SAVINGS) WHETHER BASED ON CONTRACT, TORT
 * OR ANY OTHER LEGAL THEORY, EVEN IF MENTOR GRAPHICS OR ITS LICENSORS HAVE
BEEN
 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.
```

```
 * <p>
 */
package com.example.plugin.bridge;

import com.mentor.chs.api.IXConnector;
import com.mentor.chs.api.IXValue;
import com.mentor.chs.plugin.IXApplicationContext;
import com.mentor.chs.plugin.IXHarnessPlugin;
import com.mentor.chs.plugin.IXIntegratorPlugin;
import com.mentor.chs.plugin.IXLogicPlugin;
import com.mentor.chs.plugin.IXTopologyPlugin;
import com.mentor.chs.plugin.changemanager.IXBridgeExportPostProcessor;

import java.io.DataOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.LineNumberReader;
import java.io.OutputStream;
import java.util.ArrayList;
import java.util.Collection;
import java.util.HashSet;
import java.util.Set;

/**
 * Sample plugin to demonstrate bridge export post processing.
 * <p/>
 * For each connector in the design, its user properties are written as a
comment line in the exported DSI file
 */
public class DSIConnectorUserPropertyExportProcessor
            implements IXBridgeExportPostProcessor, IXHarnessPlugin,
IXIntegratorPlugin, IXTopologyPlugin, IXLogicPlugin
{

      private static final String DSI = "DSI";
      private static final String COMMENT = "! ";
      private static final String SECTION = "%";
      private static final String UTF8 = "UTF8";
      private static final int COMPONENT_SECTION_COUNT = 5;

      /**
       * Return the description of the plugin
```

```
       *
       * @return String type - description of the plugin
       */
      public String getDescription()
      {
            return "For each connector, its user properties are written as
comments in the exported DSI file";
      }

      /**
       * Return the name of the plugin
       *
       * @return String type - name of the plugin
       */
      public String getName()
      {
            return "DSI Connector User Property Export Processor";
      }

      /**
       * Return the version of the plugin
       *
       * @return String type - version of the plugin
       */
      public String getVersion()
      {
            return "1.0";
      }

      @Override
      public boolean isAdaptorFormatHandledByPlugin(String
adaptorFormatDescription)
      {
            return adaptorFormatDescription.contains(DSI);
      }

      @Override
      public boolean process(String adaptorFormatDescription, InputStream
adaptorData,
                  OutputStream processedAdaptorData, IXApplicationContext
applicationContext) throws Exception
      {
```

```
            Collection<String> commentLines =
getConectorPropertiesAsCommentLines(applicationContext);
            StringBuilder modifiedData =
createModifiedAdaptorDataString(adaptorData, commentLines);
            boolean modifiedAdaptorData = !commentLines.isEmpty();
            if (modifiedAdaptorData) {
                  processOutputStream(modifiedData, processedAdaptorData);
            }
            return modifiedAdaptorData;
      }

      private void processOutputStream(StringBuilder modifiedData,
OutputStream processedAdaptorData)
                  throws IOException
      {
            DataOutputStream dos = new DataOutputStream(processedAdaptorData);
            try {
                  dos.writeBytes(modifiedData.toString());
            }
            finally {
                  dos.close();
            }
      }

      private StringBuilder createModifiedAdaptorDataString(InputStream
adaptorData, Collection<String> commentLines)
                  throws IOException
      {
            StringBuilder modifiedData = new StringBuilder();
            InputStreamReader reader = new InputStreamReader(adaptorData,
UTF8);
            LineNumberReader lineReader = new LineNumberReader(reader);
            try {
                  writeModifiedDataLines(commentLines, lineReader,
modifiedData);
            }
            finally {
                  lineReader.close();
                  reader.close();
            }
            return modifiedData;
      }
```

```
	private void writeModifiedDataLines(Collection<String> commentLines,
LineNumberReader lineReader,
		StringBuilder modifiedData) throws IOException
	{
		boolean propsWritten = false;
		int sectionCount = 0;
		for (String line = lineReader.readLine(); line != null; line =
lineReader.readLine()) {
			if (line.startsWith(SECTION)) {
				sectionCount++;
			}
			modifiedData.append(line).append("\n");
			if (!propsWritten && sectionCount ==
COMPONENT_SECTION_COUNT) {
				modifiedData.append(COMMENT).append("\n");

	modifiedData.append(COMMENT).append(getName()).append(" -
").append(getDescription())
					.append("\n");
				for (String comment : commentLines) {
					modifiedData.append(comment).append("\n");
				}
				modifiedData.append(COMMENT).append("\n");
				propsWritten = true;
			}
		}
	}

	private Collection<String>
getConectorPropertiesAsCommentLines(IXApplicationContext applicationContext)
	{
		Set<IXConnector> connectors = new HashSet<IXConnector>();
		if (applicationContext != null &&
applicationContext.getCurrentDesign() != null &&

	applicationContext.getCurrentDesign().getConnectivity() != null) {
			connectors =
applicationContext.getCurrentDesign().getConnectivity().getConnectors();
		}
		Collection<String> commentLines = new ArrayList<String>();
		for (IXConnector connector : connectors) {
			Set<IXValue> properties = connector.getProperties();
			StringBuilder propsLine = new StringBuilder();
```

```
                    if (!properties.isEmpty()) {

      propsLine.append(COMMENT).append(connector.getAttribute("Name")).append(
" -> ");
                    }
                    for (IXValue prop : properties) {
                          propsLine.append("\"").append(prop.getName())
                                      .append("\" =
\"").append(prop.getValue()).append("\"; ");
                    }
                    commentLines.add(propsLine.toString());
            }
            return commentLines;
      }
}
```